

Python with Parallel Programming

Data Serialization

- Serialization using JSON
- Serialization using Pickle

Multiprocessing

- Running Two Simple Processes
- Using Pool and Map

Multithreading

- Basics of multithreading
- Communicating between threads
- Creating a worker pool
- Advanced use of multithreads
- Stoppable Thread with a while Loop

Processes and Threads

- Global Interpreter Lock
- Running in Multiple Threads
- Running in Multiple Processes
- Sharing State Between Threads
- Sharing State Between Processes

Getting Started with Parallel Computing and Python

The parallel computing memory architecture

- SISD
- MISD
- SIMD
- MIMD

Memory organization Shared memory

Distributed memory

o Massively parallel processing o A cluster of workstations o The heterogeneous architecture Parallel programming models

- The shared memory model
- The multithread model
- The message passing model
- The data parallel model

How to design a parallel program

- Task decomposition
- Task assignment
- Agglomeration
- Mapping o Dynamic mapping
 - Manager/worker
 - Hierarchical manager/worker
 - Decentralize

How to evaluate the performance of a parallel program

- Speedup
- Efficiency
- Scaling
- Amdahl's law
- Gustafson's law

Python in a parallel world

Introducing processes and threads

Start working with processes in Python

Start working with threads in Python

Thread-based Parallelism

- Using the Python threading module
- Define a thread
- Determine the current thread
- Use a thread in a subclass
- Thread synchronization with Lock and RLock
- Thread synchronization with RLock
- Thread synchronization with semaphores
- Thread synchronization with a condition
- Thread synchronization with an event
- Using the with statement
- Thread communication using a queue
- Evaluating the performance of multithread applications

Process-based Parallelism

- Spawn a process
- Name a process
- Run a process in the background
- Kill a process
- Use a process in a subclass
- Exchange objects between processes
 - o Using queue to exchange objects
 - o Using pipes to exchange objects
- Synchronize processes
- Manage a state between processes
- Use a process pool

- Using the mpi4py Python module
- Point-to-point communication
- Avoiding deadlock problems
- Collective communication using broadcast
- Collective communication using scatter
- Collective communication using gather
- Collective communication using Alltoall
- The reduction operation
- How to optimize communication

Asynchronous Programming

- Using the concurrent.futures Python modules
- Dealing with the process and thread pool
- Event loop management with Asyncio
- Handling coroutines with Asyncio
- Task manipulation with Asyncio
- Dealing with Asyncio and Futures

Distributed Python

- Using Celery to distribute tasks
- Create a task with Celery
- Scientific computing with SCOOP
- Handling map functions with SCOOP
- Remote Method Invocation with Pyro4
- Chaining objects with Pyro4
- Developing a client-server application with Pyro4
- Communicating sequential processes with PyCSP

- Using MapReduce with Disco
- .Remote procedure call with RPyC

GPU Programming with Python

- Using the PyCUDA module
- A hybrid programming model
- The kernel and thread hierarchy
- Build a PyCUDA application
- Understanding the PyCUDA memory model with matrix manipulation
- Kernel invocations with GPUArray
- Evaluating element-wise expressions with PyCUDA
- The MapReduce operation with PyCUDA
- GPU programming with NumbaPro
- Using GPU-accelerated libraries with NumbaPro
- Using the PyOpenCL module
- Build a PyOpenCL application
- Evaluating element-wise expressions with PyOpenCl
- Testing your GPU application with PyOpenCL